

COMP2001: ARTIFICIAL INTELLIGENCE

METHODS – LAB 2 EXERCISES

RECOMMENDED TIMESCALES

This lab exercise is designed to take no longer than two hours, and it is recommended that you complete this within a week. The contents of this exercise were covered in lecture 2.

GETTING SUPPORT

The computing exercises have been designed to facilitate flexible studying and can be worked through in your own time or during timetabled lab hours. It is recommended that you attend the labs, even if you have completed the exercises in your own time, to discuss your solutions and understanding with one of the lab support team and/or your peers. It is entirely possible that you might make a mistake in your solution which leads to a false observation and understanding.

OBJECTIVES

The purpose of this lab exercise is to gain experience in implementing and comparing some “hill-climbing” local search heuristic that were covered in lecture 2 and deepen your understanding of experimental design.

- COMP2001 framework background including evaluating objective value of solutions.
- Implementation and evaluation of Steepest Descent (SDHC) and Davis’s Bit (DBHC) Hill Climbing heuristics.

LEARNING OUTCOMES

By completing this lab exercise, you should meet the following learning outcomes:

- Students should gain experience implementing some local search heuristics for solving combinatorial optimisation problems.
- Students should understand that no single heuristic performs better than any other given heuristic through practical experimentation and analysis.
- Students should be able to analyse the behaviour of different heuristics and critique their individual strengths and weaknesses.

TASK 1 – IMPORTING LAB EXERCISE FILES

Download the source code for this lab exercise from the Moodle page. You should find various files related to the hill climbing heuristics, and the exercise runner configuration, and the exercise runner itself. Drag the **heuristics** folder into your IDE so that it is a direct subfolder of “com.aim”. The “Lab2ExercisesTestFrameConfig” and “Lab2ExercisesRunner” classes should be placed within the “runner” package alongside the ones from lab 1.

TASK 2 – STEEPEST DESCENT AND DAVIS’S BIT HILL CLIMBING

Template code is provided for you in `DavissBitHillClimbing.java` and `SteepestDescentHillClimbing.java`. Your task is to implement these hill climbing heuristics by writing the code to **perform a single pass** of the heuristic within the method `“public void applyHeuristic(SAT oProblem, int iSolutionIndex)”`.

Pseudocode is supplied as the header to each apply heuristic method in the respective class defining the heuristic. Pseudocode is not code, and you will need to use the appropriate methods within the COMP2001 API to implement the heuristics. Some of these methods do not exist, for example `createRandomPermutation()`, and you should implement these yourself.

When implementing both heuristics, we would like you to perform the hill climbing step as accepting only improving moves. That is, the candidate solution/bit flip is accepted if and only if the objective value of the process results in an improved solution, otherwise the previous solution state should be reverted to. This can have the added benefit of preventing the search from becoming stuck on plateau’s/neutral spaces/shoulder regions in the search landscape.

$$f(s_{i+1}) \leftarrow \begin{cases} f(s'_i) & f(s'_i) < f(s_i) \\ f(s_i) & \text{otherwise} \end{cases}$$

TASK 3 – FORMATIVE ASSESSMENT (MOODLE QUIZ)

The questions in the “Lab 2 Formative Assessment” require you to perform a number of experiments using your implementations of the hill-climbing algorithms. Answer the questions by configuring the `Lab2ExercisesRunner` and `Lab2TestFrameConfig` classes as appropriate, running the algorithms, and analysing the resulting outputs. You may want to keep a copy of the results presented after running each of those experiments – instructions are given at the end of this document.

Note that feedback for open textbox questions will be provided no later than the next lab and will address common themes at the cohort level. If you wish to discuss individual answers, please ask one of us during one of the lab sessions.

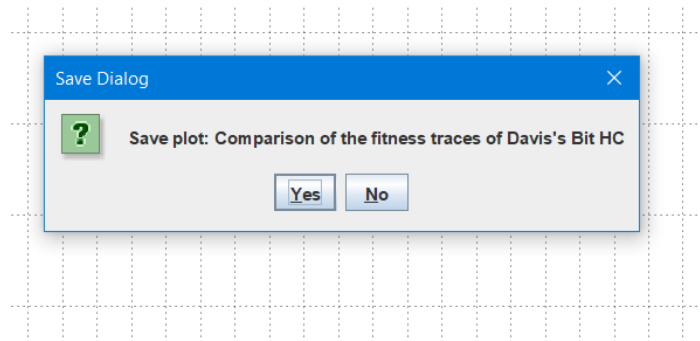
ADDITIONAL INFORMATION AND TIPS

Within the test frame configuration for this lab, there are two parameters that you will need to change:

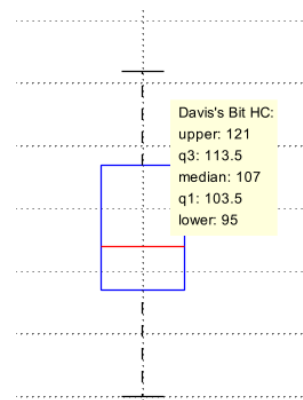
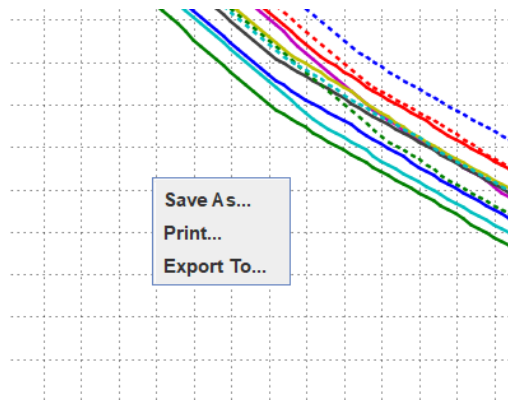
1. The MAX-SAT instance to be solved (referenced by instance ID). There are a total of 12 problem instances which you can solve by changing this between 0 and 11.
2. The total execution time. You should vary this between 1 second and 20 seconds as necessary.

When you run an experiment and once this has finished, you will be presented with a fitness plot for each heuristic which shows the search trajectory for each trial, and a boxplot of the best solutions found by each heuristic in each trial to allow you to compare their performance. You

can save the images of the plots as a PDF document by clicking on the window itself. (For the top-most window, you should click off it and select it again for the save dialog to appear).



Alternatively, you can save the plot and data by right clicking on the plots and choosing “Save As...” to save the plot, or “Export To...” to save the data as CSV files. Note that this saves each series as a separate file so you will need to merge these files and name them appropriately before exporting the data of another plot.



The boxplot does not support saving data however you can hover over each box to read off the data.